

Ex. 1

Algo 1: (Méthode 1).

for $k = 1:i-1$

for $l = 1:j-1$

$B[k, l] = A[k, l]$

end

$1:j-1$

end

for $k = 1:i-1$

for $l = j+1:n$

$B[k, l-1] = A[k, l]$

end $j:n-1$

$j+1:n$

end

for $k = i+1, n$

for $l = 1, j-1$

$B[k-1, l] = A[k, l]$

end

end.

for $k = i+1, n$

for $l = j+1, n$

$B[k-1, l-1] = A[k, l]$

end

end.

return B

Boucle ① $(i-1) \times (j-1)$ affectations.

①

② $(i-1) \times (n-j)$ affectations

③ $(n-i) \times (j-1)$ affectations

④ $(i-1) \times (n-j)$ affectations

Complexité en temps / arithmétique

$$C_t = (n-1)^2$$

$$= O(n^2).$$

②

Complexité en espace (mémoire)

$(n-1) \times (n-1)$ mots mémoires par B

$n \times n$ mots mémoire par A

$$C_m = 2n^2 - 2n + 1 = O(n^2)$$

③

④

Ex. 1.

Algo 1: (Méthode 2)

Entrée : $A \in \mathbb{R}^{n \times n}$, i , j

Sortie : $B \in \mathbb{R}^{n-1 \times n-1}$

$r = 1$

for $l = 1 \dots n$
if $l = i$ *test 2x plus cher
que affecter.*
sauter

end

$m = 1$

for $c = 1 \dots n$

if $c = j$
sauter

end

$B[l, c] = A[r, m]$

$m = m + 1$

end

$r = r + 1$

end

return B

Ex.2

Algo 2 : $\det(A)$

Entrées : $A \in \mathbb{R}^{n \times n}$.

Sorties : d déterminant de A .

DSL: domain spécifique Lanaguge

◦ matlab libre graduit → Octave

◦ Scilab : plus bas niveau.

$$\|C\|_2 \leq \sqrt{\|C\|_1 \|C\|_\infty}$$

$$\|C\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^n |C_{ij}|$$

$$\|C\|_\infty = \max_{1 \leq i \leq n} \sum_{j=1}^n |C_{ij}|$$

$$\|C\|_F = \sqrt{\sum_{i=1}^n \sum_{j=1}^n |a_{ij}|^2}$$